

Question 1:

Check the current SELinux status on your Linux system using the `getenforce` and `sestatus` commands, and write down what each output means.

Answer:

- **Command 1:** `getenforce`
 - **Output:** Displays the current operational state directly (Enforcing, Permissive, or Disabled).
 - **Meaning:**
 - Enforcing: SELinux security policy is active, strictly enforced, and unauthorized actions are blocked and logged.
 - Permissive: SELinux is active but does not block unauthorized actions; it only logs policy violations for auditing/troubleshooting.
 - Disabled: SELinux is completely turned off and no policies are loaded or enforced.
- **Command 2:** `sestatus`
 - **Output:** Provides a detailed overview of the SELinux environment:
 - SELinux status: Shows if SELinux is enabled or disabled.
 - SELinuxfs mount: Location where the SELinux virtual filesystem is mounted (typically `/sys/fs/selinux`).
 - SELinux root directory: Location of configuration files (`/etc/selinux`).
 - Loaded policy name: The active policy profile (commonly `targeted`).
 - Current mode: The runtime operational state (enforcing or permissive).
 - Mode from config file: The default mode loaded at system boot (`/etc/selinux/config`).

Question 2:

Switch SELinux to permissive mode temporarily using the `setenforce` command, then try creating a new file in `/var/www/html` and note any differences in error messages or permissions compared to enforcing mode.

Hint: Use `setenforce 0` for permissive and `setenforce 1` to return to enforcing.

Answer:

- **Step 1: Switch to permissive mode**

`sudo setenforce 0`

Verify mode: `getenforce` (*Outputs: Permissive*)

- **Step 2: Create a test file**

`sudo touch /var/www/html/test_file.txt`

- **Step 3: Return to enforcing mode**

`sudo setenforce 1`

Verify mode: `getenforce` (*Outputs: Enforcing*)

- **Observations and Differences:**

- **In Permissive Mode (setenforce 0):** Operations that violate SELinux policies (such as an unauthorized service daemon attempting to write to `/var/www/html`) are allowed to succeed, but an AVC (Access Vector Cache) audit denial entry is logged to `/var/log/audit/audit.log`. Standard POSIX permissions (file read/write bits) still apply.
- **In Enforcing Mode (setenforce 1):** Any action that violates SELinux policy rules is immediately blocked by the Linux kernel with a Permission denied error, even if traditional POSIX permissions (`chmod 777`) allow it.

Question 3:

List all files in `/var/www/html` and use the `ls -Z` command to display their SELinux security contexts. Change the context of one file to `httpd_sys_content_t` using `chcon` and verify the change.

Hint: Use `chcon --type=httpd_sys_content_t filename`

Answer:

- **Step 1: View existing SELinux contexts**

`ls -lZ /var/www/html`

(Output shows fields formatted as `user:role:type:level`, e.g., `unconfined_u:object_r:httpd_sys_content_t:s0 test_file.txt`)

- **Step 2: Change context type to `httpd_sys_content_t` using `chcon`**

`sudo chcon --type=httpd_sys_content_t /var/www/html/test_file.txt`

(Or short form: `sudo chcon -t httpd_sys_content_t /var/www/html/test_file.txt`)

- **Step 3: Verify the updated context**

`ls -lZ /var/www/html/test_file.txt`

(Verification confirmed: The third field of the context reflects `httpd_sys_content_t`, allowing web server daemons such as Apache/Nginx to read and serve the file).

Question 4:

A user is unable to upload images to a folder for their food review blog (like Zomato) hosted in `/srv/foodie/images`. Use the `audit2why` and `audit2allow` tools to analyze SELinux denial logs and suggest a solution to allow uploads.

Hint: Check `/var/log/audit/audit.log` for recent denials.

Answer:

- **Step 1: Analyze recent denials using `audit2why`**

Filter web daemon denials (`httpd_t`) from the audit log and pipe to `audit2why`:

`sudo grep "httpd" /var/log/audit/audit.log | audit2why`

- **Diagnostic Output Meaning:** `audit2why` explains that the web server process running under type `httpd_t` was denied write access to files/directories labeled with default or generic contexts (e.g., `var_t` or `default_t`).

- **Step 2: Inspect suggested policy rules using `audit2allow`**

`sudo grep "httpd" /var/log/audit/audit.log | audit2allow`

- It shows the missing rule allowing `httpd_t` to append/write to the directory context.

- **Step 3: Recommended Permanent Solution (Best Practice):**

Instead of compiling a custom module, apply the built-in writable SELinux context `httpd_sys_rw_content_t` to the upload folder so the web service has permanent read-write access:

1. Add persistent file context mapping:

`sudo semanage fcontext -a -t httpd_sys_rw_content_t "/srv/foodie/images(/.*)?"`

2. Apply the context changes to the filesystem:

`sudo restorecon -Rv /srv/foodie/images`

3. Verify the applied context:

```
ls -ldZ /srv/foodie/images
```

(Displays httpd_sys_rw_content_t, resolving the upload block).